Formative 2 • Group 9 • 13 November 2026

Multimodal Biometric Product Recommender

Face recognition, voiceprint verification and product recommendation
in one command-line authorisation flow

Team member	Area owned
Hassan Adelani Luqman	Data merge, cleaning and EDA; product recommendation model
Emma Tiffany Umwari	Image pipeline; facial recognition model
Parfait Shema Semana	Audio pipeline; voiceprint verification model
Peter Michael Angelo Rucakibungo	System integration; CLI app; unauthorised-attempt simulation

GitHub repository https://github.com/Hassan-Adelani-Luqman/g9-biometric-product-recommender

Demo video **[add the demo video link here before submitting]**

Summary. We built a command-line system that authorises a purchase with two biometrics and then shows a personalised product recommendation. A photo is checked by a facial recognition model (logistic regression on pretrained SFace embeddings, with a cosine-similarity gate for strangers). If the face is accepted, the product recommendation model (logistic regression trained on merged social-media and transaction data) predicts the customer's next category, but the result is *held*. A voice clip is then verified by an XGBoost voiceprint model, and the voice must belong to *the same member the face step recognised*. Only then is the product displayed. The face model scores 100% accuracy on held-out photos and rejects all 27 real non-member images; the voice model reaches 65% held-out accuracy (chance 25%) and rejects 97% of other members' voices; the product model beats the class-prior baseline on accuracy in every evaluation view (27% vs 23% on the test set, 29% on unseen customers). The app passes all six required demo scenarios. We report the main limitation openly: with one recording per phrase, the voice model cannot reliably reject a *new* speaker, so strangers are stopped by the face step.

Contents

1 Introduction and system overview **3**
- 1.1 Problem . . . 3
- 1.2 Architecture . . . 3

2 Data merge, cleaning and EDA (Hassan) **4**
- 2.1 Sources and data issues . . . 4
- 2.2 Cleaning decisions . . . 4
- 2.3 Join strategy and post-merge validation . . . 4
- 2.4 Feature engineering (leakage-safe) . . . 5
- 2.5 Exploratory data analysis . . . 5

3 Image pipeline (Emma) **8**
- 3.1 Collection protocol . . . 8
- 3.2 Augmentation . . . 9
- 3.3 Feature extraction . . . 9

4 Audio pipeline (Parfait) 10
4.1 Collection protocol . . . 10
4.2 Augmentation . . . 12
4.3 Feature extraction . . . 12

5 Models and evaluation 13
5.1 Product recommendation model (Hassan) . . . 13
5.2 Facial recognition model (Emma) . . . 15
5.3 Voiceprint verification model (Parfait) . . . 16
5.4 Summary of the three models . . . 17

6 Multimodal decision logic and system simulation (Peter) 17
6.1 Decision logic . . . 17
6.2 Scenario results . . . 18
6.3 Robustness . . . 19

7 Limitations and future work 20

8 Links and reproduction 20

9 Team contributions 21

1 Introduction and system overview

1.1 Problem

The brief asks for a user identity and product recommendation system that combines three models. A user presents a face and a voice; the system must deny access if either biometric fails, and otherwise show a product recommendation for that user. Every step must be explainable and evaluated. Our system follows the flow diagram in the brief exactly:

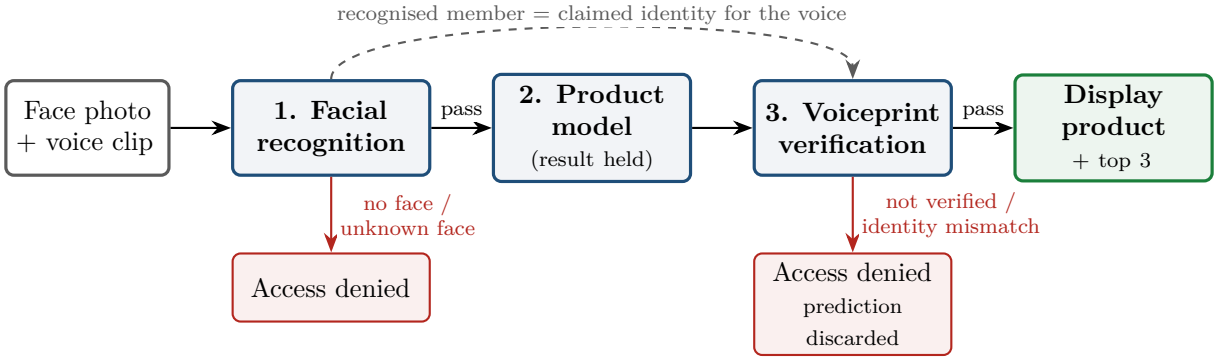


Figure 1: System flow. The product prediction is computed after the face step but shown only if the voice step confirms the same member (dashed arrow: the cross-modal check).

1.2 Architecture

All logic lives in importable Python modules. The notebooks and the CLI app call the same functions, so a photo or clip is processed identically during training and at login. Shared paths, member IDs and the member-to-customer mapping are in `src/config.py`; tuned thresholds are saved next to each model in `models/*_meta.json` and read by the app, never hard-coded.

Table 1: Components, code and outputs.

Stage	Code	Output	Owner
Merge, clean, engineer	`src/data_merge.py`, notebook 01	`data/processed/merged_customer_data.csv` (150 × 35)	Hassan
Image features	`src/image_pipeline.py`, notebook 02	`features/image_features.csv` (135 × 194)	Emma
Audio features	`Audio_features/audio_pipeline.py`, `Audio_features/audio_processing.ipynb`	`Audio_features/data/audio_features.csv` (48 × 70)	Parfait
Product model	`src/train_product_model.py`, notebook 04 Part A	`models/product_model.joblib` + meta	Hassan
Face model	`src/train_face_model.py`, notebook 04 Part B	`models/face_model.joblib` + meta	Emma
Voice model	`src/train_voice_model.py`, notebook 04 Part C	`models/voice_model.joblib` + meta	Parfait
CLI app and demo	`src/app.py`, notebook 05	6 demo scenarios, decision figure	Peter

The full pipeline can be rebuilt from a fresh clone with the commands in Section 8. The trained models are committed, so the app runs straight after `pip install -r requirements.txt`.

2 Data merge, cleaning and EDA (Hassan)

Evidence: `notebooks/01_data_merge_eda.ipynb`, `src/data_merge.py`, figures `reports/figures/eda_*.png`.

2.1 Sources and data issues

Table 2: The two source files and what inspection revealed.

File	Rows	Key	Issues found
`customer_social_profiles.csv`	155	`customer_id_new` (`A178`)	5 exact duplicate rows; 41 of 84 customers have 2–6 profiles (not one row per customer)
`customer_transactions.csv`	150	`customer_id_legacy` (`178`)	10 missing `customer_rating` (6.7%); `purchase_date` stored as text

The keys use different formats but the same 100–199 numbering, so stripping the `A` prefix gives a common integer `customer_id`. The overlap is partial: 61 customers appear in both files, 14 transaction customers have no profile, and 23 profile customers have no transactions.

2.2 Cleaning decisions

Table 3: Cleaning decisions and their justification.

Issue	Decision	Reason
Key mismatch (`A178` vs `178`)	Strip `A`, cast to int	Same numbering in both files
5 duplicate profile rows	Dropped (155 → 150)	Exact repeats add nothing and bias the aggregates
Several profiles per customer	Aggregated to one row per customer (counts, mean/max engagement and interest, sentiment score and mix, per-platform counts, dominant platform)	Avoids a many-to-many join that would duplicate transactions
10 missing ratings	Customer's own median (6), else global median 3.0 (4); flag `rating_was_missing`	Rating behaviour is personal; the flag keeps the imputation visible
`purchase_amount` outliers	IQR rule, flag only; 0 found (fences −135 and 703, data 51–495)	Every row matters at $n = 150$
Types	`purchase_date` → datetime; numeric casts; domain checks (5 platforms, 3 sentiments, valid score ranges)	Enforced in code

2.3 Join strategy and post-merge validation

The prediction unit is a transaction, so the merged table must have exactly one row per transaction. We compared three joins:

Table 4: Join options. The chosen join keeps every labelled transaction exactly once.

Join	Rows	Unique transactions	Problem
Raw profiles, inner (many-to-many)	213	117	Transactions duplicated, labels double-counted
Aggregated profiles, inner	117	117	Drops 33 transactions (22%) with no profile
Aggregated profiles, left (chosen)	**150**	**150**	None: every transaction kept once

Aggregating first makes the right-hand side unique per customer, so the join is many-to-one; pandas enforces this with `validate="many_to_one"`. The 23 profile-only customers have no purchases and therefore no label, so they could not be training rows under any join. For the 33 transactions without a profile we distinguish *absence of activity* from *unknown level*: counts and shares are set to 0, sentiment to neutral (0), engagement and interest to the median of customers with profiles, and a `has_social_profile` flag lets the model tell imputed rows from real ones.

Post-merge checks (all 7 pass, enforced by `validate_merge`): row count preserved (150 = 150); `transaction_id` unique; same transaction set as the source; 0 nulls after imputation; target has no nulls; aggregates match the raw profile rows for every customer; unmatched customers truly lack profiles. A spot check confirms it: customer 187's five transactions each carry engagement 85.67, the mean of the three raw profiles $(97 + 71 + 89)/3$.

2.4 Feature engineering (leakage-safe)

- **Date:** `purchase_month`, `purchase_dayofweek`, `is_weekend`.
- **Customer history**, computed from each customer's *earlier* transactions only (sorted by date, expanding window shifted by one): number of earlier purchases, first-purchase flag, average earlier amount and rating, days since the previous purchase, and the most frequent earlier category (`cust_prev_category`, ties to the most recent). A row's own category is never used to build its own features.
- **Leakage:** `purchase_amount`, `amount_bucket` and `customer_rating` describe the purchase being predicted (a rating is given after buying), so they are kept for EDA but excluded from the model.

Output: `merged_customer_data.csv`, 150 rows × 35 columns, 0 nulls, with a reload check.

2.5 Exploratory data analysis

Variable types and summary statistics. The 35 columns are 3 identifiers, 1 nominal target, and continuous, discrete, binary and nominal features. Scales differ widely (amount 51–495, rating 1–5, engagement 50–99), so features are standardised for logistic regression. Skew is mild ($|\text{skew}| < 0.5$) for every original variable, so no transforms are needed. The history is thin: 30 of 75 customers bought once, and exactly half of all transactions (75/150) are a customer's first purchase.

Table 5: Summary statistics of key numeric variables (150 transactions).

Variable	Mean	Std	Min	Median	Max	Skew
`purchase_amount`	280.8	121.3	51	274	495	0.06
`customer_rating`	2.97	1.18	1.0	3.0	5.0	−0.07
`avg_engagement`	74.5	11.2	50	75.8	99	−0.47
`avg_purchase_interest`	3.12	0.92	1.0	3.25	4.8	−0.48
`sentiment_score`	0.08	0.66	−1	0.00	1	−0.11
`cust_prev_txn_count`	0.81	1.03	0	0.5	5	1.47

Target balance. The five categories are close to balanced: Sports 35 (23%), Electronics 34 (23%), Clothing 28, Groceries 28, Books 25 (17%); the max/min ratio is only 1.4. No resampling is needed, but we use stratified splits, report macro F1, and compare against a 23.3% majority-class baseline (20% for random guessing).

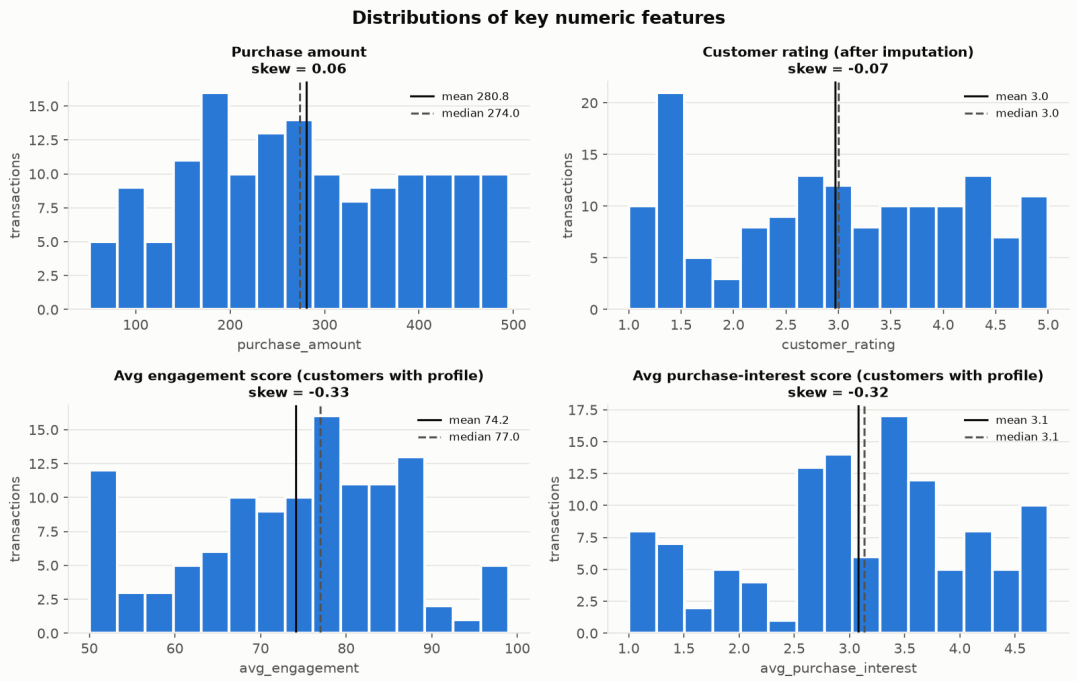


Figure 2: Distributions of the key numeric features. Engagement and interest use customers with a profile only, so imputed medians do not distort the shape.

Interpretation. Purchase amounts are spread roughly uniformly between 51 and 495 with no long tail. Ratings cover the whole 1–5 scale (mean and median 3.0) with a small cluster at very low values; real review data usually piles up at 4–5 stars, so this flat shape suggests the data is synthetic. Engagement and purchase interest are mildly left-skewed, pulled down by a group of low-engagement customers near 50. All features are bounded and roughly symmetric, which is an early warning that they may carry little signal about the category.

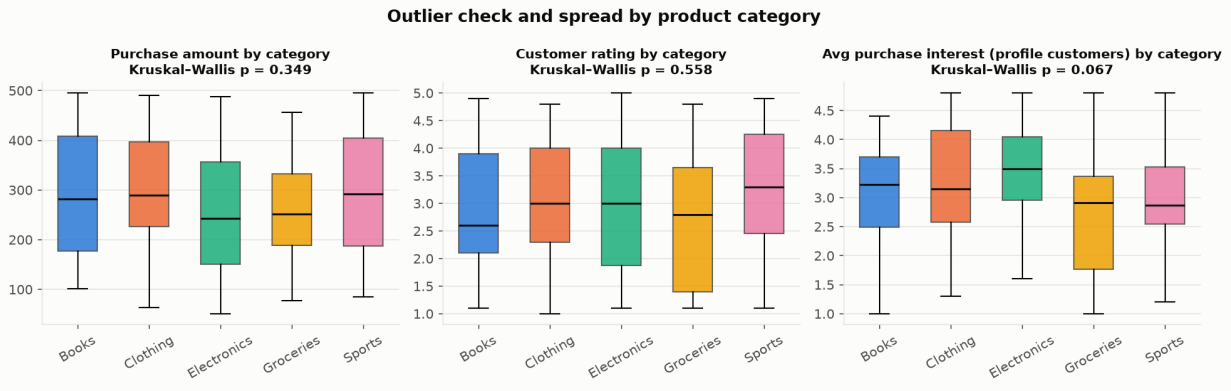


Figure 3: Purchase amount, rating and purchase interest by product category (box plots).

Interpretation. No point lies beyond the whiskers, globally or within any category, which supports flagging rather than dropping outliers. Category differences are small and the boxes overlap heavily: a Kruskal–Wallis test finds no significant difference in amount ($p = 0.35$) or rating ($p = 0.56$), and purchase interest is borderline ($p \approx 0.07$). Including the 33 imputed rows would create 19 false "outliers" because they all share the value 3.25, which shows why the `has_social_profile` flag matters.

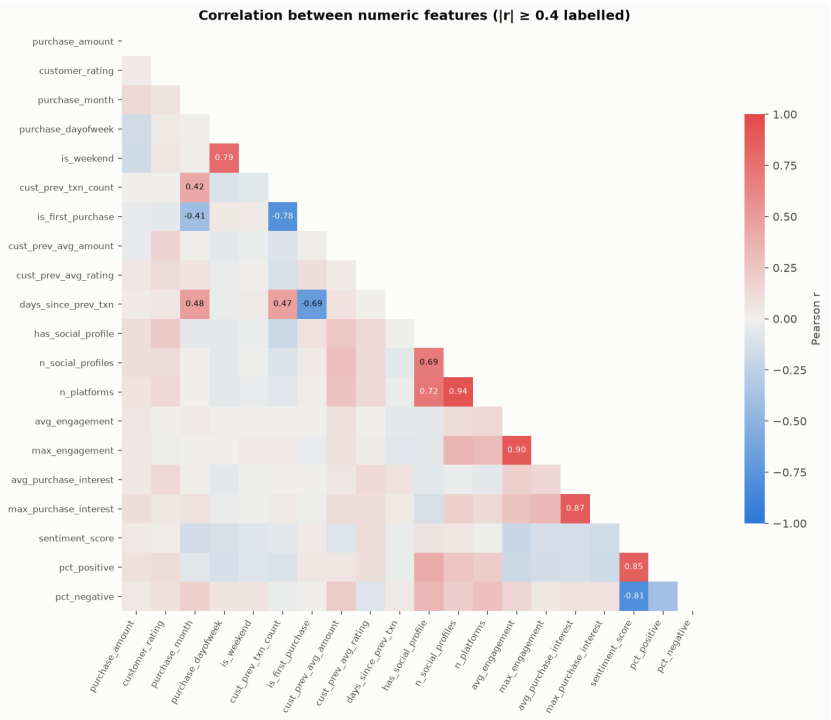


Figure 4: Correlation matrix of the numeric features (blue–red diverging scale, grey at 0).

Interpretation. The strongly correlated pairs are engineered summaries of the same profiles: `n_platforms`–`n_social_profiles` ($r = 0.94$), `avg_`/`max_engagement` (0.90), `avg_`/`max_purchase_interest` (0.87) and `sentiment_score`–`pct_positive` (0.85). For logistic regression we keep one feature per group. The original variables are almost independent of each other (e.g. rating–amount $r = 0.03$).

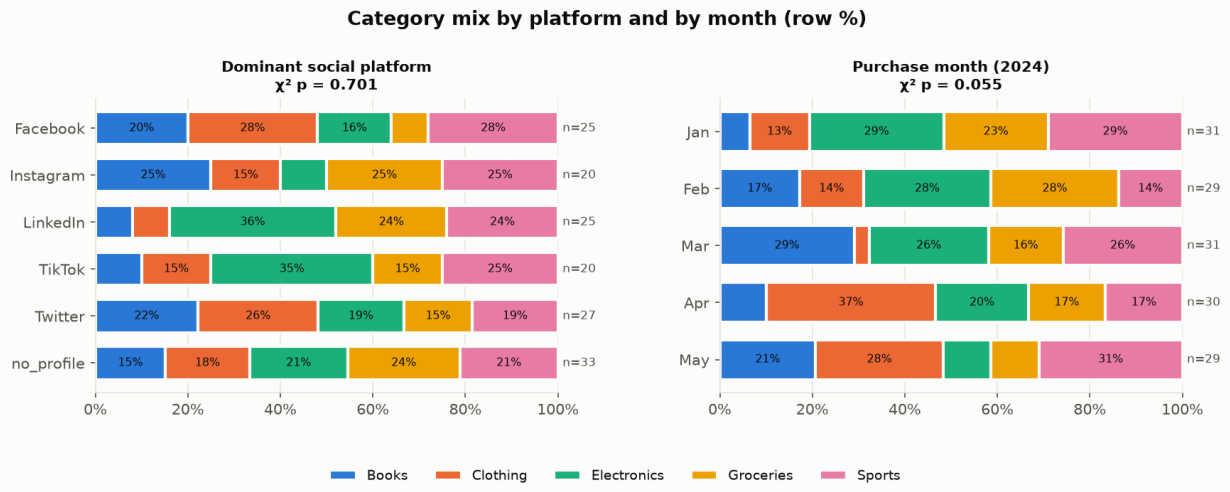


Figure 5: Category mix by dominant social platform and by purchase month (row percentages, n per row on the right).

Interpretation. LinkedIn- and TikTok-dominant customers buy more Electronics (35–36%), but the platform mix is not significantly different overall (χ^2 $p = 0.70$). Customers without a profile buy the same mix as everyone else ($p \approx 0.93$), so the left join's imputed rows are not a biased subgroup. Month is the clearest pattern: April is dominated by Clothing (37%), and seasonality is borderline significant ($p = 0.055$). Purchase history matters as a *transition* rather than a repeat: returning customers buy their usual category again only 20% of the time, but, for example, customers whose history is mostly Groceries bought Sports next in 10 of 21 cases (χ^2 $p = 0.015$).

EDA takeaways for modelling. (1) No single numeric feature separates the categories, so modest accuracy is expected. (2) `cust_prev_category` and `purchase_month` are the most promising features. (3) Redundant pairs are reduced for linear models. (4) Half the rows are first purchases, so `is_first_purchase` must stay. (5) Post-purchase columns are excluded to avoid leakage.

3 Image pipeline (Emma)

Evidence: `notebooks/02_image_processing.ipynb`, `src/image_pipeline.py`, `features/image_features.csv`, figures `reports/figures/img_*.png`.

3.1 Collection protocol

Each of the 4 members took one front-facing photo for each of three expressions (*neutral*, *smiling*, *surprised*): 12 images. Raw phone files were normalised by `src/ingest_biometrics.py`: renamed to `<member>_<expression>_<take>.jpg`, rotated upright from the orientation flag, and **stripped of all metadata**, because one set of photos carried GPS coordinates. All 12 files share one naming pattern and format (JPEG, RGB); resolutions range from 478×1080 to 2316×3088, so every photo is downscaled to a longest side of 1024 px before detection. All members consented to their samples being stored in the repository.

Observations. Every member shows three clearly different expressions. Backgrounds, lighting and framing differ between members (plain wall, living room, ceiling; tight portrait crops vs. wider shots). This is realistic for login attempts, but it means features computed on the *whole* photo could identify the setting rather than the person. All features are therefore computed on the detected, aligned face only.

Detection and alignment. Two pretrained OpenCV Zoo models are used: **YuNet** detects the face and 5 landmarks, a landmark-based similarity transform aligns a 112×112 crop (eyes and mouth in fixed positions), and **SFace** computes a 128-dimensional identity embedding. YuNet finds exactly

one face in all 12 photos with confidence 0.86–0.95 (acceptance threshold 0.60).

3.2 Augmentation

Each photo receives **8 augmentations** (the rubric asks for at least 2), each simulating a condition the login camera could face. They are applied to the full photo *before* detection, so the detector and aligner must cope with them exactly as at login, and they are deterministic (fixed noise seed), so rerunning the pipeline reproduces the CSV exactly.

Table 6: Image augmentations and their effect on the embedding (cosine similarity to the original photo's embedding, 12 photos each).

Augmentation	Parameters	Simulates	Mean cos	Min cos
`rotate_p15`, `rotate_m15`	$\pm 15°$, reflected border	tilted head or phone	0.94	0.83
`flip`	horizontal mirror	front-camera mirroring	0.94	0.91
`grayscale`	BGR $\rightarrow$ grey $\rightarrow$ 3 channels	colourless camera, white-balance loss	0.93	0.88
`brighter`	contrast $\times 1.25$, $+30$	overexposure	0.94	0.87
`darker`	contrast $\times 0.6$, -10	dim room	0.95	0.88
`blur`	Gaussian 9×9	out of focus, motion	0.97	0.95
`noise`	Gaussian $\sigma = 15$, fixed seed	low-light sensor noise	0.86	0.74

Interpretation. Detection is fully robust: a face is found in all 108 member versions, with mean confidence 0.90–0.92 for every augmentation. The embedding barely moves for blur, lighting, flip and rotation (mean similarity 0.93–0.97), because alignment undoes tilt and SFace was trained to ignore lighting and pose. Noise is the hardest case (mean 0.86, worst 0.74), so noisy or low-light inputs are where the face model is most likely to struggle; training on these noisy copies is what makes it tolerant.

3.3 Feature extraction

`features/image_features.csv` has **135 rows** $\times$ **194 columns**: 15 source photos (12 member + 3 non-member) $\times$ (original + 8 augmentations), with faces detected in 135/135 rows and 0 nulls. Validation checks (row count, unique IDs, unit-length embeddings, histograms summing to 1) all pass.

Table 7: Feature groups in `image_features.csv`.

Group	Columns	Captures
Metadata	8	labels; `source_image` is the grouping key that keeps augmented copies with their original
SFace embedding	128 (`emb_0`–`emb_127`)	identity (face shape and proportions)
Colour histogram	48 (16 bins $\times$ B, G, R)	skin tone, lighting, camera colour
LBP texture histogram	10 (uniform LBP, $P = 8$, $R = 1$)	local skin and hair texture

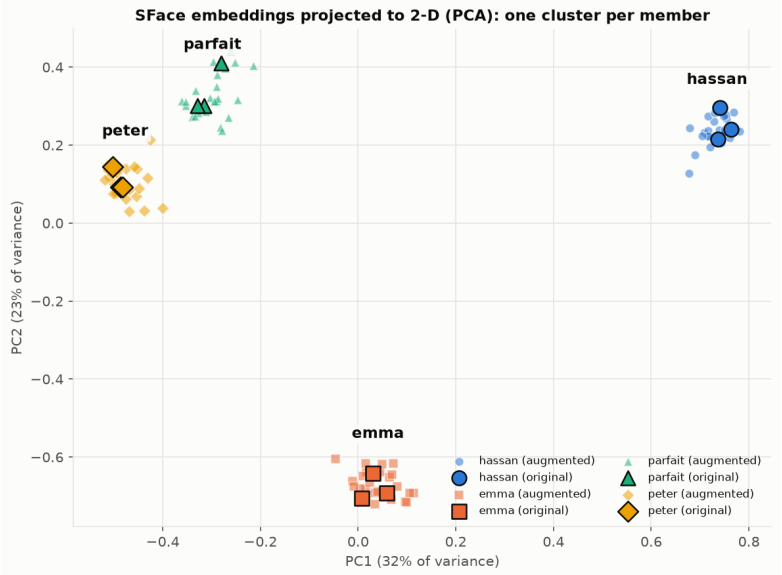


Figure 6: PCA of the SFace embeddings. Large outlined markers are originals; small markers are their augmentations.

Do the features separate the members? Each member forms a separate cluster and augmented versions stay close to their originals. In the full 128-dimensional space, two images of the same person always score higher than any pair of different people (same person: mean similarity 0.81, minimum 0.47; different people: mean 0.17, maximum 0.46; 0 overlapping pairs). Holding out each photo with all its augmentations, a nearest-centroid rule identifies the member 100% of the time from embeddings, against 78% for LBP texture and 71% for colour histograms. Because each member photographed themselves in one room, colour partly encodes *where* the photo was taken, so the face model relies on the embedding.

Non-member photos. Three photos of one non-member (from a public stock-photo source) in `data/unauthorized/faces/` are processed identically and labelled `unknown`; they are never used for training. Their highest similarity to any member image is 0.34, below every same-person pair (0.47).

4 Audio pipeline (Parfait)

Evidence: `Audio_features/audio_processing.ipynb`, `Audio_features/audio_pipeline.py`, `Audio_features/prepare_audio.py`, `Audio_features/data/audio_features.csv`, figures in `Audio_features/figures/`.

4.1 Collection protocol

Each member recorded the two phrases *"Yes, approve"* and *"Confirm transaction"* once on their own phone: 8 recordings. The phones produced different formats (`.m4a`, `.aac`, `.mp4`), so `prepare_audio.py` converts every recording to **16 kHz mono WAV** named `<member>_<phrase>_<take>.wav`; the originals are kept in `Audio_features/data/audio/original/`. On loading, each clip is trimmed of leading and trailing silence (30 dB) and peak-normalised. Normalisation matters because the phones recorded at very different volumes (peaks of 0.09 for Emma and 0.65 for Hassan); without it the model could learn loudness instead of voice. After trimming, clips last 1.2–3.5 s.

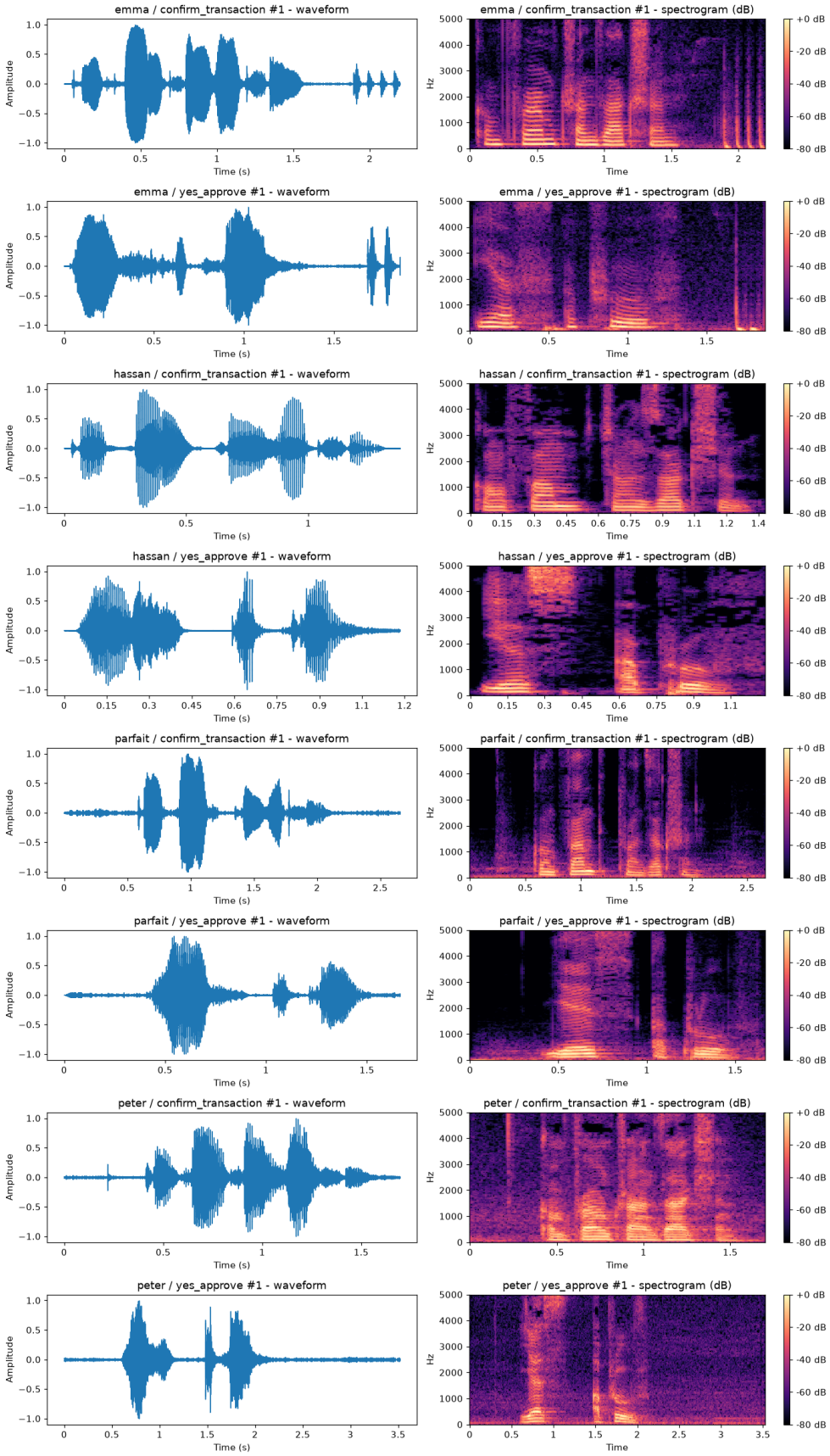


Figure 7: Waveform and spectrogram of every member recording (4 members × 2 phrases).

Interpretation. *Waveforms:* each syllable appears as a burst of energy, and "Confirm transaction" has more bursts than "Yes, approve" (e.g. 4–5 vs. 2–3 for Emma). No clip is flat-topped, so none

is clipped. Emma's clips end with short clicks, and Peter's "Yes, approve" has about 1.5 s of low background sound after speaking. *Spectrograms:* the horizontal stripes are harmonics of each speaker's pitch. Peter has the lowest voice (about 90 Hz), so his stripes are packed closest together; Hassan has the most high-frequency energy (a bright 4–5 kHz band on the "s" of "yes"). These timbre differences are what the MFCC features capture. *Noise:* Hassan's and Parfait's recordings have a dark (clean) background between words, while Emma's and Peter's show a grey haze of background noise.

4.2 Augmentation

Each recording receives **5 augmentations**: pitch shift +2 and −2 semitones, time stretch ×1.15 and ×0.85, and a combined version with background noise at 15 dB SNR, 0.6× gain and a 100 ms shift. These simulate the speaker on a different day, speaking at a different speed, or using a worse microphone.

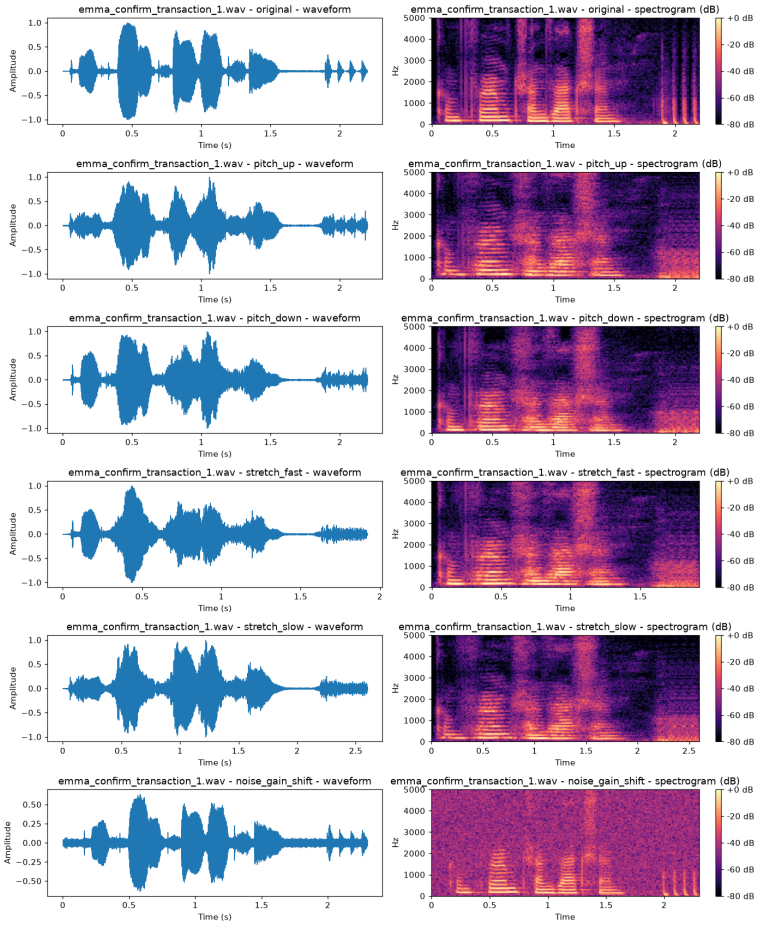


Figure 8: One recording and its augmented versions (waveforms and spectrograms).

4.3 Feature extraction

For every clip (original and augmented) the mean and standard deviation over time of the following features are saved to `audio_features.csv`: **48 rows** (8 recordings × 6 versions) × 70 columns (6 metadata + 64 features), 0 missing values.

Table 8: Audio features (mean and standard deviation of each over time).

Feature	Why it identifies a speaker
13 MFCCs + 13 delta-MFCCs	shape of the vocal tract (timbre) and how it moves
Spectral roll-off (85%)	how much high-frequency energy the voice has
Spectral centroid and bandwidth	"brightness" and spread of the voice
Zero-crossing rate	noisiness and fricatives
RMS energy	loudness profile
Fundamental frequency (YIN)	the speaker's pitch

Clip duration is deliberately excluded: it describes the phrase, not the speaker.

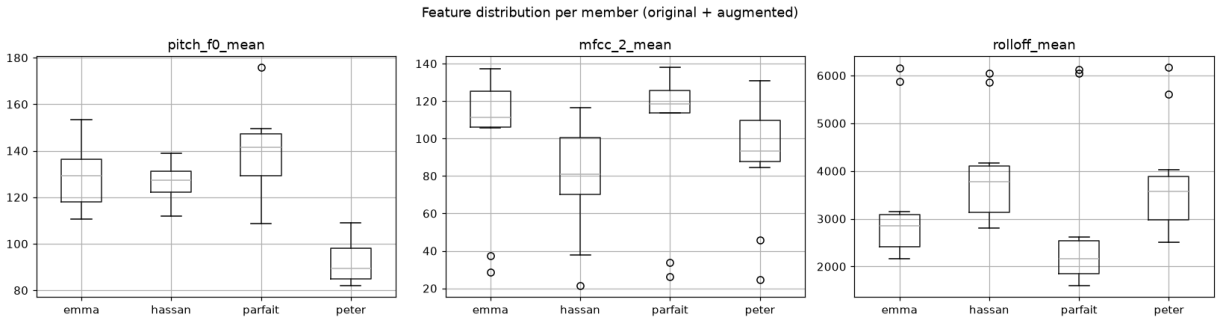


Figure 9: Distribution of selected audio features per member, across original and augmented clips.

Interpretation. Where a member's boxes do not overlap with the others' (especially pitch and the low MFCCs), that voice is easy to separate. Wide boxes show the effect of augmentation: pitch shifting spreads `pitch_f0_mean` on purpose, so the model cannot rely on pitch alone.

Non-member clips. Four clips from two people outside the team are used only for testing: two from Ketia, a consenting non-member recorded for the project (`Audio_features/data/audio/unauthorized/`), and two from another consenting non-member (`data/unauthorized/voices/`).

5 Models and evaluation

Evidence: `notebooks/04_model_training_evaluation.ipynb`, `src/train_*.py`, `models/*_meta.json`. Every model reports **accuracy**, **macro F1** and **log loss** on data it was not trained on, and every split keeps related rows together so that no augmented copy or repeat customer leaks between training and evaluation.

5.1 Product recommendation model (Hassan)

Task. Predict the 5-class `product_category` of a customer's next purchase from information available *before* the purchase (date, earlier-purchase history, social profile). Post-purchase, redundant and identifier columns are excluded (Section 2.4), leaving 20 numeric and 2 categorical features.

Protocol. (1) Stratified 80/20 split: 120 training rows, 30 held-out test rows, touched once. (2) Three algorithms (multinomial logistic regression, random forest, XGBoost) × three feature sets (all 22 features; previous category + month; previous category only), each tuned on a small grid with 5-fold stratified CV scored on log loss. (3) Every tuned model re-evaluated with repeated stratified 5-fold CV ($5 \times 10 = 50$ folds), because one 5-fold run on 120 rows is very noisy. (4) Lowest mean CV log loss wins; it must also beat the class-prior baseline. (5) Customer-grouped CV on all 150 rows checks generalisation to customers never seen in training. (6) The chosen configuration is refit on all 150 rows.

Table 9: Product model comparison: repeated stratified CV on the training split (mean of 50 folds). Lower log loss is better.

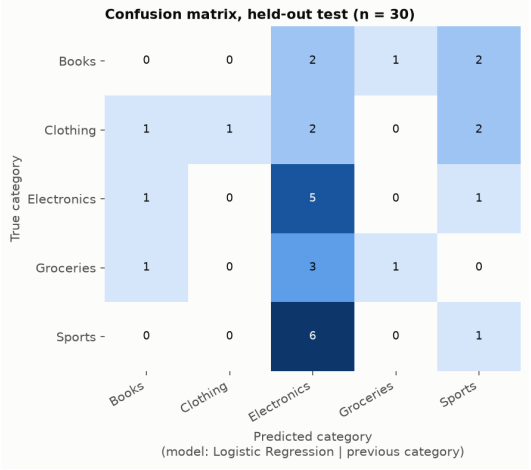
Algorithm	Features	Accuracy	Macro F1	Log loss	Top-3 acc.
Logistic Regression	**previous category**	**0.282**	0.241	**1.575**	0.707
Logistic Regression	all features	0.274	0.268	1.577	0.710
Random Forest	previous category	0.272	0.228	1.579	0.702
Random Forest	all features	0.253	0.242	1.583	0.711
XGBoost	previous category	0.276	0.205	1.593	0.663
Logistic Regression	prev. category + month	0.245	0.231	1.593	0.668
Random Forest	prev. category + month	0.231	0.212	1.602	0.663
XGBoost	prev. category + month	0.223	0.192	1.692	0.663
XGBoost	all features	0.270	0.256	1.795	0.680
Baseline (class prior)	–	0.208	0.069	1.604	0.650

Every model beats the baseline on accuracy, but only by a few points, and the fold-to-fold spread (±7–10 accuracy points) is larger than the gaps between models; this is why log loss over 50 folds is the selection criterion. Logistic regression on the previous category alone has the lowest log loss and highest accuracy; the all-features logistic regression is almost tied. Generalisation to unseen customers breaks the tie:

Table 10: Customer-grouped 5-fold CV (no customer in both training and validation).

Algorithm	Features	Accuracy	Macro F1	Log loss
Logistic Regression	**previous category**	**0.294**	**0.267**	**1.595**
Logistic Regression	all features	0.213	0.194	1.680
Random Forest	all features	0.167	0.156	1.644
XGBoost	all features	0.213	0.190	1.962
Baseline (class prior)	–	0.227	0.074	1.602

The all-features models *collapse* on unseen customers (17–21%, below the 23% baseline): their CV score came from recognising customers by their constant social-profile values, not from patterns that transfer. Permutation importance on customer-grouped folds agrees: `cust_prev_category` is the most useful feature, date features help slightly, and most social-profile features are at or below zero (`sentiment_score` clearly hurts). The simpler model holds up, so it was chosen.



Held-out test set (30 rows). Accuracy 0.267, macro F1 0.225, log loss 1.655, top-3 accuracy 0.567, against 0.233 accuracy and 1.601 log loss for the baseline. The 95% bootstrap intervals (about 10–43% vs. 10–40%) overlap almost completely, and the test log loss is slightly worse than the baseline's, so a 30-row test set cannot prove the gain on its own; the 50-fold CV and grouped CV estimates are the more reliable ones.

The confusion matrix shows the main weakness: 18 of 30 test predictions are Electronics. Half the rows are first purchases with no history, and the model gives them all the most common first-purchase category (Electronics), which yields high Electronics recall (5/7) and none for Books.

Figure 10: Product model confusion matrix on the 30-row held-out test set.

What the model learned. With one input, the model is a smoothed table of $P(\text{next category} \mid$

previous category). It reproduces the real habits in the data but shrinks them towards 20% (regularisation $C = 0.3$; each row rests on only 12–21 transactions): e.g. Clothing → Clothing 36% (raw 50%), Groceries → Sports 33% (raw 48%). The app therefore shows the **top 3** categories with probabilities, which is the honest way to present a model that is rarely confident.

Table 11: Final product model (logistic regression on previous category, $C = 0.3$) in all three evaluation views.

View	Accuracy	Macro F1	Log loss	Baseline accuracy
CV on training split (50 folds)	0.282	0.241	1.575	0.208
Held-out test (30 rows)	0.267	0.225	1.655	0.233
Unseen customers (grouped CV)	0.294	0.267	1.595	0.227

5.2 Facial recognition model (Emma)

Task. Identify which of the 4 members is in a photo and **reject anyone else**. **Data:** 108 member rows (12 photos × 9 versions). **Split:** leave-one-source-photo-out (12 folds): a photo and its 8 augmented copies are always held out together, so every fold tests an expression the model has never seen for that member. **Candidates:** logistic regression, linear SVM (Platt-calibrated) and random forest, each on the embedding alone (128) and on embedding + colour/texture (186). **Selection:** lowest held-out log loss; differences below 0.001 are ties won by the smaller feature set.

Table 12: Face model candidates (leave-one-source-photo-out, 108 held-out predictions).

Model	Features	Accuracy	Macro F1	Log loss
Logistic Regression | embedding	128	1.000	1.000	0.0072
Logistic Regression | embedding + colour/texture	186	1.000	1.000	0.0069
SVM (linear) | embedding + colour/texture	186	1.000	1.000	0.0559
SVM (linear) | embedding	128	1.000	1.000	0.0852
Random Forest | embedding	128	1.000	1.000	0.2371
Random Forest | embedding + colour/texture	186	1.000	1.000	0.2651

Every candidate identifies every held-out photo correctly, including rotated, flipped, darkened, blurred and noisy copies: the pretrained SFace network does the hard work, and the classifier only separates 4 well-separated clusters. Log loss separates the candidates; logistic regression on the embedding is chosen (adding colour/texture changes log loss by less than 0.001).

Rejecting strangers. A 4-class classifier always names one of the 4 members, so it needs a separate stranger check. We compared two scores using *simulated strangers* (leave-one-member-out: each member is removed from training and their 27 rows are scored against the other three):

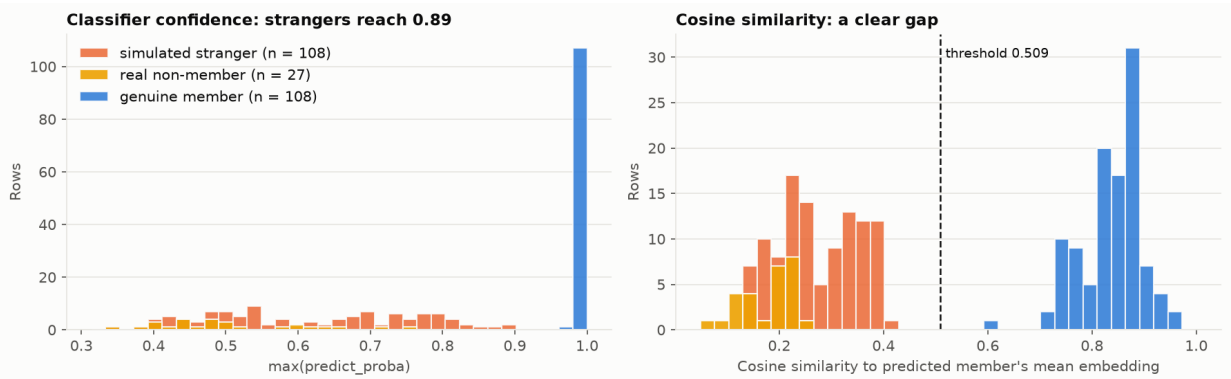


Figure 11: Classifier confidence (left) cannot separate genuine members from strangers; cosine similarity to the predicted member's enrolled mean embedding (right) separates them completely.

Interpretation. Classifier confidence is the wrong signal: softmax always spreads 100% across the known members, so a stranger who looks a little like one member gets 0.6–0.9. Similarity to the enrolled face separates the groups with a clear gap: every held-out genuine photo scores ≥ 0.610 and every simulated stranger ≤ 0.409. The threshold **0.509** sits in the middle of the gap. On **real** non-member photos, never used to choose the threshold, all **27/27** rows are rejected (highest similarity 0.25), even though the classifier alone would have named Emma or Hassan with up to 0.75 confidence. `identify_face()` applies both checks: the classifier names the closest member and the similarity check confirms it.

5.3 Voiceprint verification model (Parfait)

Task. *Verification*, not identification: the face step makes a claim ("this is Parfait") and the voice must confirm it. A clip is accepted for member m only if the classifier's top prediction is m *and* $P(m \mid \text{clip}) \geq$ threshold. **Split:** leave-one-recording-out (8 folds), with augmented copies held out with their source. Because each member recorded each phrase once, every fold trains on the speaker's *other* phrase only, which makes this a strict text-independent test. **Candidates:** logistic regression (Parfait's first model), random forest and XGBoost on the 64 features; lowest held-out log loss wins.

Table 13: Voice model candidates (leave-one-recording-out, 48 held-out predictions). Chance: accuracy 0.25, log loss 1.39.

Model	Accuracy	Macro F1	Log loss
XGBoost	**0.646**	**0.641**	**0.946**
Logistic Regression	0.417	0.415	1.136
Random Forest	0.375	0.364	1.305

XGBoost is well above chance but far below the face model. Each fold must recognise a voice from a phrase the model never heard that person say, and MFCC statistics from 1–3 s of a single phrase describe the words almost as much as the speaker. Per member, Parfait is recognised most reliably (11/12 held-out rows, F1 0.92) and Hassan least (F1 0.46; half of Hassan's rows are mistaken for Emma). A pretrained speaker-embedding network (WeSpeaker ResNet34) was also tried and was not clearly better on clips this short, so the simpler features were kept.

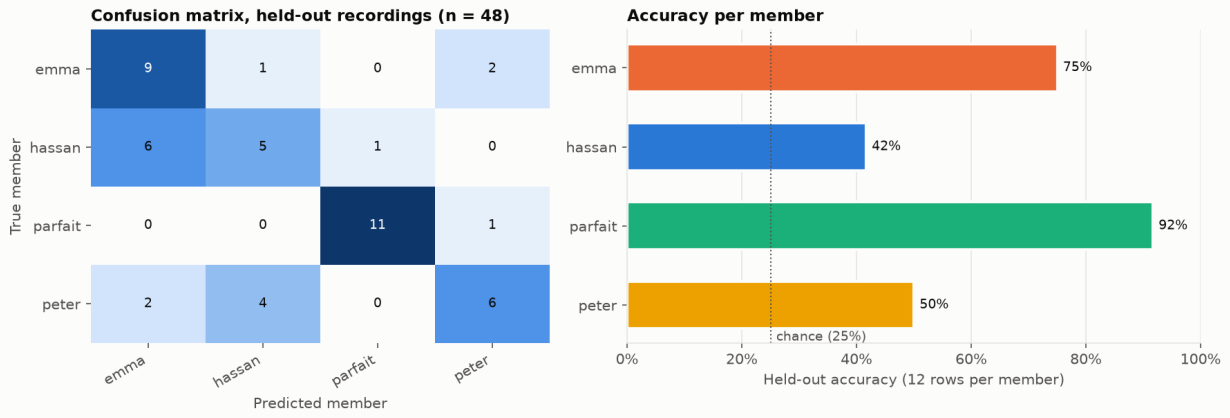


Figure 12: Voice model: held-out confusion matrix and per-member results.

Threshold. Every held-out clip yields three kinds of verification attempt: *genuine* (claims its own speaker, 48), *other member* (claims one of the other three, the app's identity-mismatch case, 144) and *simulated stranger* (leave-one-member-out, 144). The threshold minimises *false-reject rate* + 2 × *false-accept rate*, because approving a payment for the wrong person is worse than asking a member to repeat the phrase.

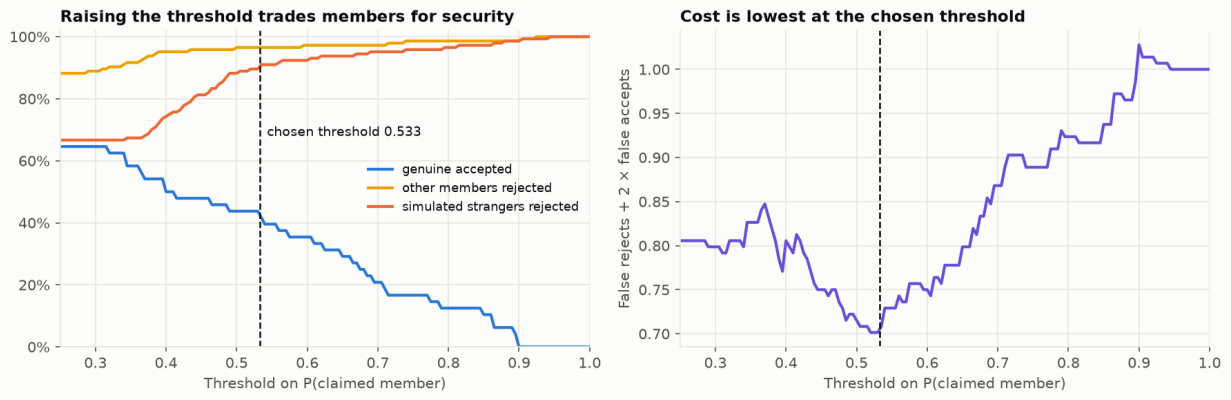


Figure 13: Raising the voice threshold trades genuine acceptance for security (left); the weighted cost is lowest at the chosen threshold 0.533 (right).

Interpretation. At **0.533** the model rejects **96.5% of other members' voices** and 90.3% of simulated strangers, but accepts only 43.8% of genuine clips it has never heard. The 2:1 cost deliberately errs towards security. However, the 4 real non-member clips are **not** rejected: each sounds confidently like one member (Ketia like Parfait at 0.86–0.93, the other non-member like Hassan at 0.58–0.85). Adding an `unknown` class trained on one stranger did not transfer to the other stranger (0–25% rejected) and lowered genuine acceptance. **Conclusion:** on its own the voice model reliably catches the *wrong member's* voice but cannot tell a *new* speaker from a member. In the full system strangers are stopped at the face step; Section 7 discusses this limitation and its fix.

5.4 Summary of the three models

Table 14: All three models on held-out data (face: leave-one-photo-out; voice: leave-one-recording-out; product: 30-row test set).

Model	Algorithm	Acc.	Macro F1	Log loss	Rejection / notes
Face	Logistic Regression (SFace emb.)	1.000	1.000	0.007	similarity ≥ 0.509; 100% genuine accepted; 100% of simulated and 27/27 real non-member rows rejected
Voice	XGBoost (64 features)	0.646	0.641	0.946	$P \geq 0.533$ and match the face; 96.5% of other members rejected; 0/4 real non-member clips rejected
Product	Logistic Regression (prev. category)	0.267	0.225	1.655	baseline 0.233 / 1.601; 0.294 accuracy on unseen customers

6 Multimodal decision logic and system simulation (Peter)

Evidence: `src/app.py`, `notebooks/05_system_demo.ipynb`, `reports/figures/system_demo_decisions.png`, demo video.

6.1 Decision logic

The app chains the three models exactly as in Figure 1:

1. **Face.** `identify_face(photo)` must detect a face *and* find it close to a member's enrolled

face (cosine similarity ≥ 0.509). Otherwise access is denied and nothing else runs.

2. **Product (held).** The recognised member is mapped to their customer record (`config.MEMBER_TO_CUSTOMER`: Hassan → 187, Emma → 101, Parfait → 114, Peter → 120; team members are not in the transaction data, so each is mapped to a real customer with transactions and a profile). `predict_for_customer(id)` computes the recommendation, which is **held** and not shown.
3. **Voice, cross-modal.** `verify_voice(clip, expected_member=face_member)` must find that the voice belongs to **the same member the face step recognised**, with $P \geq 0.533$. A voice that sounds like a different member is denied as an *identity mismatch*; the held prediction is discarded.
4. **Display.** Only now is the product shown, with its top-3 probabilities.

The cross-modal check is the core of the multimodal logic: someone holding a member's photo still needs that member's voice, and two members cannot combine their biometrics (scenario 5). Holding the prediction means a failed attempt reveals nothing about the customer.

CLI. `python src/app.py --demo` runs the six required scenarios; `--face <photo> --voice <clip>` runs one transaction; with no arguments the app is interactive and asks for the voice clip only after the face is accepted. Missing files, wrong file types, corrupt images and photos with no face produce a clear message instead of a traceback. Exit codes are 0 (granted), 1 (denied) and 2 (bad input). Loading the models takes about 4 s once; each transaction then takes 0.2–0.6 s on a laptop.

6.2 Scenario results

Table 15: The six demo scenarios (`python src/app.py --demo`). Face: cosine similarity to the recognised member (threshold 0.509). Voice: probability that the voice is the face's member (threshold 0.533).

#	Face input	Voice input	Face	Voice heard as (P)	Result
1	Hassan, neutral	Hassan, "Yes, approve"	0.97	Hassan (0.95)	**Groceries shown**
2	Emma, smiling	Emma, "Confirm transaction"	0.92	Emma (0.97)	**Books shown**
3	Non-member	(not reached)	0.22	–	**denied** at face
4	Parfait, neutral	Non-member	0.93	Hassan (0.07)	**denied** at voice (mismatch)
5	Parfait, surprised	Peter, "Yes, approve"	0.90	Peter (0.01)	**denied** at voice (mismatch)
6	Peter, rotated −15°	Peter + noise (20 dB SNR)	0.92	Peter (0.97)	**Sports shown**

All 6 scenarios behave as required (**6/6**). Scenario 6 generates its inputs at run time with the same augmentation code as the pipelines, so they are new inputs, not files from the training set.

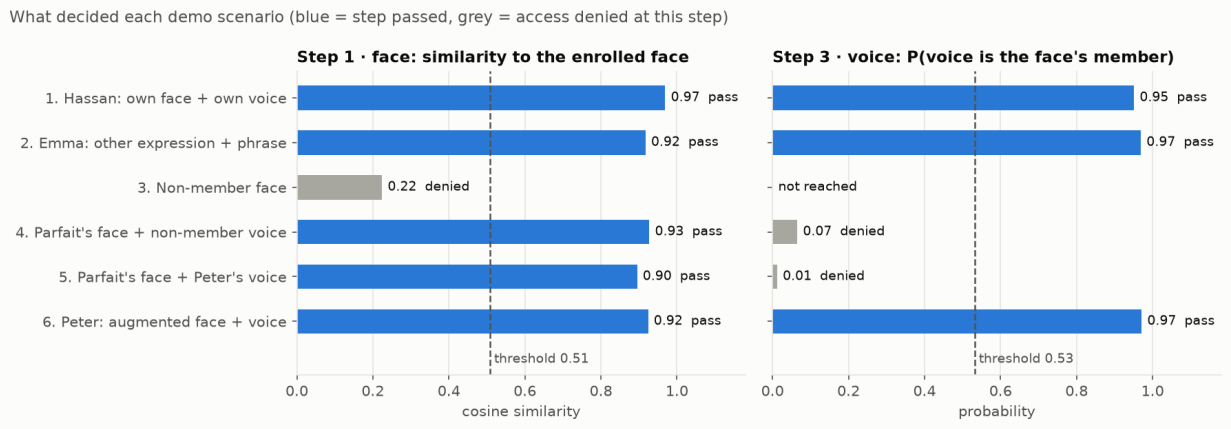


Figure 14: The scores behind each decision: face similarity (left) and the probability the voice belongs to the face's member (right). Blue bars pass; grey bars are denied. Dashed lines are the thresholds.

Interpretation. Every member photo scores 0.90–0.97, far above the face threshold, and the non-member scores 0.22, so the face margin is wide on both sides. Genuine voices give their own member 0.95–0.97. In scenarios 4 and 5 the voice gives Parfait almost no probability because the model hears Hassan and Peter, so both are denied as identity mismatches and the held product is never shown. Scenario 3 never reaches the voice step.

An excerpt of the demo output, with the per-step timings removed for width (the app prints `[OK]` / `[X]` in terminals without emoji support):

```
Scenario 1: Authorised member (Hassan)
  Face : data/images/hassan/hassan_neutral_01.jpg
  Voice: Audio_features/data/audio/raw/hassan_yes_approve_1.wav
[1/3] Facial recognition...       [OK] hassan (similarity 0.97 >= 0.51)
[2/3] Product recommendation...   computed for customer 187, held until the voice check passes
[3/3] Voiceprint verification...  [OK] hassan (confidence 0.95 >= 0.53)
[OK] ACCESS GRANTED. Recommended product for Hassan Adelani Luqman (customer 187): Groceries
     1. Groceries    30.2%
     2. Books        22.6%
     3. Electronics  17.3%

Scenario 5: Parfait's face + Peter's voice
  Face : data/images/parfait/parfait_surprised_01.jpg
  Voice: Audio_features/data/audio/raw/peter_yes_approve_1.wav
[1/3] Facial recognition...       [OK] parfait (similarity 0.90 >= 0.51)
[2/3] Product recommendation...   computed for customer 114, held until the voice check passes
[3/3] Voiceprint verification...  [X] identity mismatch: voice sounds like 'peter', face was 'parfait'
[X] ACCESS DENIED at the voice step. The held prediction is discarded; no product shown.
```

6.3 Robustness

- **Faces:** all 108 member images (3 photos × 4 members × 9 versions) are accepted as the right member, 100% for every augmentation including noise and ±15° rotation.
- **Voices with new background noise** (2 clips per member, 3 random seeds per level, none used in training): 100% accepted at 20 dB and 15 dB SNR for every member. At 10 dB, where the noise is nearly as loud as the voice, Hassan's clips pass 67% and Parfait's 33% of the time. Failing by rejecting is the safe direction: the member repeats the phrase and nobody else gets in. (These are new versions of recordings the model was trained on; held-out performance on new recordings is the 44% genuine acceptance in Section 5.3.)

7 Limitations and future work

- **The voice model cannot reject a new speaker.** With one recording per phrase per member, all 4 real non-member clips sound confidently like some member. Paired with Parfait's face, Ketia's voice is **wrongly approved** (heard as Parfait, 0.86–0.93); the other non-member is denied only because it sounds like Hassan (identity mismatch). Strangers are therefore stopped by the face step, which rejected 27/27 real non-member images, while the voice step reliably catches the wrong member. *Fix:* 3–4 more takes of each phrase per member, so genuine scores rise while strangers' stay low; then rerun the audio pipeline and `src/train_voice_model.py`. This is the single most useful improvement to the system.
- **Small, single-session biometric data.** 3 photos and 2 clips per member, each from one session and one phone. The face model's perfect held-out score means "works for this demo", not "solved"; other cameras, glasses or poor light would lower it. Only one non-member face and two non-member voices were available for real stranger tests.
- **No liveness or anti-spoofing.** A printed photo or a replayed recording of a member would be treated as genuine. A production system would add liveness detection (blink or head-movement challenges, randomised spoken phrases).
- **Sensitivity to heavy noise.** Voice acceptance drops at 10 dB SNR; face embeddings drift most under sensor noise.
- **Product model data.** 150 transactions, near-balanced classes and half of the rows being first purchases leave little to learn; accuracy is modestly but consistently above the baseline. First-time customers always receive the most common first purchase (Electronics). The ratings' flat distribution suggests synthetic data. More transactions per customer would sharpen the transition table and let date features contribute.
- **Simulation assumptions.** Members are mapped to real customers, and predictions are made as of 2024-05-30, the day after the data ends.

8 Links and reproduction

GitHub repository https://github.com/Hassan-Adelani-Luqman/g9-biometric-product-recommender

Demo video **[add the demo video link here before submitting]**

From a fresh clone (Python 3.12):

```
pip install -r requirements.txt
python src/data_merge.py                    # -> data/processed/merged_customer_data.csv
python src/image_pipeline.py                # -> features/image_features.csv
python Audio_features/audio_pipeline.py     # -> Audio_features/data/audio_features.csv
python src/train_product_model.py           # -> models/product_model.joblib + meta
python src/train_face_model.py              # -> models/face_model.joblib + meta
python src/train_voice_model.py             # -> models/voice_model.joblib + meta
python src/app.py --demo                    # the 6 scenarios: "6/6 scenarios behaved as expected."
```

The first run downloads the two pretrained face networks (about 39 MB, checksum-verified). All notebooks are committed with their outputs saved, so every number in this report can be checked without rerunning them.

9 Team contributions

Each member committed their own work from their own GitHub account, so the repository history shows who did what.

Table 16: Team contributions.

Team member	Tasks completed	Main files
Hassan Adelani Luqman	Project plan, repository setup and shared configuration; data cleaning, merge and validation; feature engineering; EDA; product recommendation model and its evaluation; ingestion script for the team's raw photos and recordings; face photos and voice clips	`src/data_merge.py`, `src/train_product_model.py`, `src/config.py`, `src/ingest_biometrics.py`, notebook 01, notebook 04 Part A
Emma Tiffany Umwari	Image collection standards; face detection, alignment, augmentation and feature extraction; facial recognition model, stranger-rejection threshold and evaluation; non-member face samples; face photos and voice clips	`src/image_pipeline.py`, `src/train_face_model.py`, `features/image_features.csv`, notebook 02, notebook 04 Part B
Parfait Shema Semana	Audio collection and conversion; waveform and spectrogram analysis; audio augmentation and feature extraction; voiceprint verification model, threshold and evaluation; non-member voice samples; face photos and voice clips	`Audio_features/` (`audio_pipeline.py`, `prepare_audio.py`, notebook, `audio_features.csv`), `src/train_voice_model.py`, notebook 04 Part C
Peter Michael Angelo Rucakibungo	System integration; command-line app (single, interactive and demo modes) with error handling; multimodal decision logic; unauthorised-attempt simulation and robustness tests; system demo notebook; demo video; face photos and voice clips	`src/app.py`, notebook 05, `reports/figures/system_demo_decisions.png`